

Weather Explorer

A full-stack case study: fetch historical daily weather from Open-Meteo, store the raw JSON in cloud object storage, and browse it in a dashboard.

InRisk Labs case study · 44 commits · 116 backend tests · 53 frontend tests · ~882 backend LOC · ~2205 frontend LOC · Python 3.12 / FastAPI · React 19 / Vite 8 / TypeScript 6

FILL IN BEFORE SUBMISSION

GitHub repository	github.com/sarjakmodi39/weather-explorer
Live dashboard	weather-explorer-black.vercel.app
Live API	weather-explorer-qc3x.onrender.com
Last verified live	4 August 2026 — full flow exercised against production: store → Supabase, list, read back, geocode, validation errors, and CORS accepted for the deployed origin while refusing a foreign one.
Availability note	The API runs on Render's free tier, which spins down after inactivity. The first request after an idle period can take up to ~50 seconds while the container wakes; subsequent requests are immediate. Redeployment instructions are in <code>DEPLOY.md</code> .

1 What was built

The backend exposes three endpoints matching the brief:

- `POST /store-weather-data` — validates a latitude/longitude/date-range request, fetches the matching window from Open-Meteo's historical archive API, and stores the response verbatim under a generated filename.
- `GET /list-weather-files` — lists stored files (name, size, creation time), newest first.
- `GET /weather-file-content/{file}` — returns one stored file's JSON, or a 404 for anything that isn't a name this service could itself have produced.

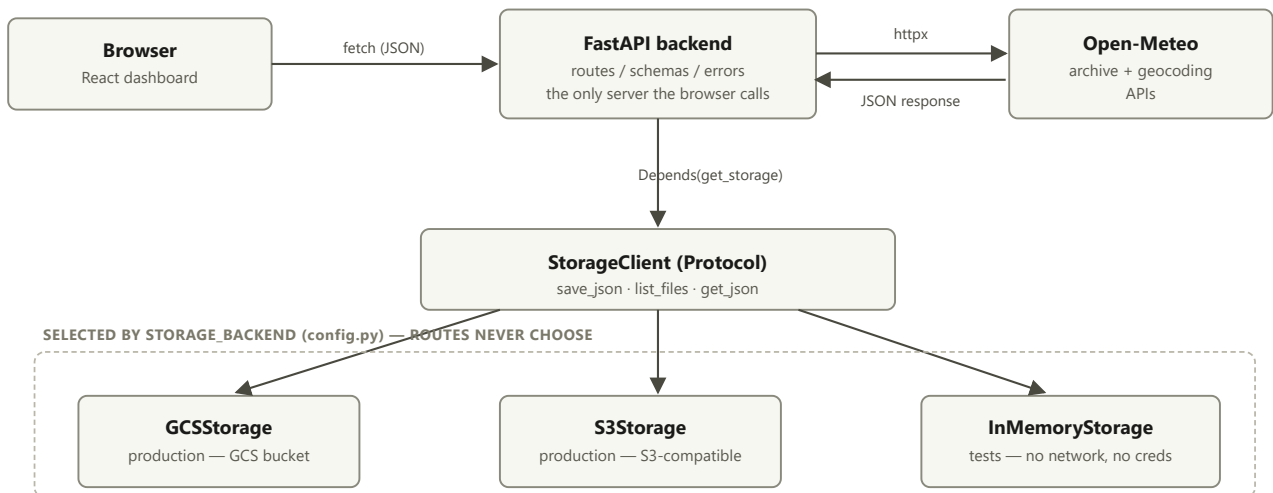
The dashboard flow: an input panel collects coordinates and a date range (with preset locations and a city-search box); submitting it stores a file and refreshes the file list; clicking a listed file loads its JSON and renders a temperature chart (or a single-day range bar, for one-day windows) plus a paginated data table, both derived from the same loaded payload.

Two additions beyond the brief's literal scope:

- **City search** (`GET /geocode`) — a thin backend proxy over Open-Meteo's own geocoding API, so a user can type a city name instead of looking up coordinates. The browser still never calls Open-Meteo directly; only the backend does.
- **An S3-compatible storage backend** (`S3Storage`), alongside `GCSStorage` — added when the deployment target moved from GCP to an S3-compatible provider (see Deployment). Both satisfy the same `StorageClient` protocol; which one is active is chosen by configuration alone.

2 Architecture

Routes stay thin (HTTP in, HTTP out); `schemas.py` owns validation; `services/` owns I/O; `naming.py` owns object-name construction and its traversal-safety check. The centrepiece is `backend/app/services/storage.py`'s `StorageClient` Protocol — a three-method surface (`save_json` , `list_files` , `get_json`) that every route depends on through FastAPI's `Depends(get_storage)` (`app/deps.py`), never on `google.cloud.storage` or `boto3` directly. Three implementations satisfy it: `GCSStorage` and `S3Storage` for production, `InMemoryStorage` for tests. Which production implementation is active is selected entirely by the `STORAGE_BACKEND` setting in `config.py` .



Why this matters, concretely: all 116 backend tests run against a dependency override

(`app.dependency_overrides[get_storage] = lambda: InMemoryStorage()`) — the whole suite needs no cloud credentials and makes no network calls except through `respx`-stubbed `httpx` . And when the deployment target changed from GCS to an S3-compatible provider (commit `1c6d8a0`), the change was contained to `services/storage.py` , `config.py` , and `deps.py` — `routes/weather.py` , `schemas.py` , and every route-level test were untouched, because they depend on the Protocol's three-method surface, not on which SDK implements it.

3 API contract

Endpoint	Request	Success	Errors
POST /store-weather-data	{latitude, longitude, start_date, end_date}	200 {"status":"ok","file":"<name>"}	400 (validation, or Open-Meteo's own rejection passed through); 502 (Open-Meteo or storage unreachable); 500 (unexpected)
GET /list-weather-files	—	200 {"files": [{"name", "size", "created_at"}]}, newest first; empty bucket → {"files": []}	502; 500
GET /weather-file-content/{file:path}	—	200 <stored JSON, unmodified>	404 for malformed or missing name — identical body either way; 502; 500
GET /geocode?q=	q (≥2 chars after trimming)	200 {"results":[{"name, admin1, country, country_code, latitude, longitude}]}; empty list for a query with no matches	400 (q too short); 502; 500

Every error response, at every one of those status codes, has the same shape: `{"status":"error","message":"<reason>"}`. This is enforced centrally in `errors.py` by three handlers — one for the app's own `AppError`, one that remaps FastAPI's default 422 validation response to 400 with a readable message, and a catch-all for anything unexpected, which logs the real exception and returns an opaque 500 body. The brief only mandates this shape for the 404 case; a single contract for every status code is worth more to the frontend's one `ApiError` class than per-case fidelity would be.

4 Design decisions and assumptions

"Range ≤ 31 days" is an inclusive day count

`schemas.py` computes `span = (end_date - start_date).days + 1` and rejects anything over `MAX_RANGE_DAYS = 31`. Jun 1 → Jul 1 is 31 days and passes; Jun 1 → Jul 2 is 32 and fails. The frontend's `validateInputs` mirrors this exact arithmetic so an obvious mistake costs no round trip.

Trade-off: the brief's phrase is genuinely ambiguous between an inclusive count and a subtraction; this reading is stated explicitly (README, spec, code comment) precisely so it doesn't need re-deriving in review.

"Full API JSON" means unmodified content, not unmodified bytes

`POST /store-weather-data` stores Open-Meteo's parsed response with no reshaping and no field stripping.

Trade-off: storage re-serializes a parsed Python dict (`json.dumps(payload)`), so the JSON *value* is byte-for-byte equivalent in content, but whitespace, key order, and number formatting from the original wire response are not preserved. If a reviewer needs the literal upstream bytes, this is the one place semantics diverge from the raw response.

Date bounds beyond the brief are validated server-side, and "today" is UTC

The brief specifies range and ordering rules but not future dates or how far back the archive goes.

`schemas.py` additionally rejects `end_date` in the future and `start_date` before 1940-01-01, evaluating "today" as `datetime.now(timezone.utc).date()` — not the server's local clock or the client's.

Trade-off: Open-Meteo rejects both of these anyway; validating locally turns an upstream 502 (its own rejection, wrapped) into a direct 400 with a specific reason. Getting the UTC part right took several attempts — see Engineering findings.

Malformed and missing filenames both return 404

`GET /weather-file-content/{file}` checks the filename against `OBJECT_NAME_PATTERN` (`naming.py`) with a strict `re.ASCII fullmatch` **before any storage call**. Anything that doesn't match — wrong shape, traversal attempt, junk — returns the identical `404 {"status":"error","message":"not found"}` a genuinely absent file gets.

Trade-off: a different code or message for "malformed" vs. "absent" would let an attacker fingerprint bucket contents by probing names. Refusing to distinguish leaks nothing, at the cost of a slightly less specific error for a legitimate typo.

Why the content route is `{file:path}`, not the brief's `{file}`

From the code, `routes/weather.py`:

```
`{file:path}` rather than the brief's plain `{file}`: a single-segment converter matches against the *decoded* path, so a traversal string like "../../etc/passwd" would fail to match this route and fall through to Starlette's generic 404 ("Not Found", capital N) instead of the "not found" below — failing our own hostile-input test. The `:path` converter matches it here so is_valid_object_name() gets a chance to reject it.
```

In plain terms: FastAPI's default single-segment path parameter cannot even capture a string containing `/` after URL-decoding — the router itself returns Starlette's generic 404 before the handler runs, so the endpoint's own validation and its own error shape never fire. Using `{file:path}` lets a hostile string reach the handler, where `is_valid_object_name()` rejects it and returns the app's uniform error body instead.

Trade-off: this is a deliberate, documented departure from the specified interface, made to *keep* the contract (one 404 shape for every kind of bad input) rather than to shortcut it — a reviewer testing the brief's literal `{file}` against a traversal string would watch it fail the very test the brief implies it should pass.

Listing is unpaginated, but deliberately cheap

`GET /list-weather-files` makes one `list_blobs` call with a `fields` mask (`items(name,size,timeCreated),nextPageToken`) so GCS returns only the three fields the response needs — no per-object metadata fetches. (The S3 implementation instead pages through `list_objects_v2`'s 1000-key truncation with a paginator, since that API truncates where GCS's does not; both stay a single logical listing operation from the caller's perspective.)

Trade-off: this assumes bucket contents stay small enough that no pagination is needed at the API layer — see Known limitations.

CORS is an explicit origin allowlist, never *

`ALLOWED_ORIGINS` in `config.py` is a comma-separated allowlist, parsed into `origin_list` and passed straight to `CORSMiddleware`.

Trade-off: requires setting the real frontend origin at deploy time (and a temporary permissive value during the two-pass deploy in `DEPLOY.md`), rather than the zero-configuration convenience of a wildcard.

The browser never calls Open-Meteo

Only `services/open_meteo.py` and `services/geocoding.py` talk to Open-Meteo; the frontend only ever calls the backend's own endpoints. Loaded files are cached in memory by name (`App.tsx`, a `useRef(new Map())`), so re-selecting an already-loaded file is instant and costs no request.

Trade-off: the cache is per-session and unbounded — acceptable for a case study's file counts, not a general-purpose cache policy.

POST /store-weather-data is public and unauthenticated

Deliberate for a case study: there are no user accounts and nothing to protect beyond the outbound Open-Meteo call and a storage write.

Trade-off: in a real deployment, the first addition would be an API key or a platform-level authenticated-invoker mechanism, plus basic rate limiting in front of this one route — it is the only one that costs an outbound network call and writes storage, so it is the one worth defending first.

5 Engineering findings

Drawn from the project's engineering ledger (`.superpowers/sdd/2026-08-01-weather-explorer/progress.md`), which records every defect found in review and the ruling that resolved it. The five below were judged the most substantive; the ledger also records roughly two dozen deferred minor findings that were consciously left as-is.

1. 500 responses bypassed CORS entirely

Symptom: a genuine server error surfaced in the browser as a blocked fetch — "Cannot reach the API. Is the backend running?" — with no indication anything had actually failed server-side.

Root cause: Starlette routes `@app.exception_handler(Exception)` to `ServerErrorMiddleware`, which sits *outside* `CORSMiddleware` in the default stack. A 500 built by that path never passes through `CORSMiddleware`, so it carries no `access-control-allow-origin` header; browsers block the response client-side, and the resulting error is indistinguishable from a network failure — pointing a debugger at connectivity instead of the logged server exception.

Fix: a `@app.middleware("http")` catch-all registered *before* `add_middleware(CORSMiddleware, ...)` in `main.py` — Starlette's `add_middleware` inserts at index 0, so registering the catch-all first makes `CORSMiddleware` the outermost layer even around an unhandled exception.

`test_unhandled_exception_still_carries_cors_header` in `test_cors.py` raises a `RuntimeError` from a throwaway route and asserts both the 500 status and the header's presence.

Found by: the final whole-branch review — no single task-scoped review had reason to combine "CORS" and "an unhandled exception" in the same test.

2. Dependency resolution runs before request-body validation

Symptom: with a misconfigured or unreachable storage backend, every `POST /store-weather-data` returned 500 — including requests with obviously invalid input (e.g. an out-of-range latitude) that should have failed validation with a 400 before storage was ever touched.

Root cause: FastAPI resolves `Depends(get_storage)` before it validates the request body. The original `GCSStorage.__init__` constructed a real `google.cloud.storage.Client()` eagerly; if that construction raised, dependency resolution failed and the request never reached pydantic's validation — a 500 silently masked what should have been a 400. Invisible to the whole test suite, because the fixtures override `get_storage` with `InMemoryStorage`, which never raises.

Fix: both `GCSStorage` and `S3Storage` are now inert at construction — the bucket name is stored, but the real client is built lazily behind a `_client` property on first actual use. Validation now always runs first; a storage failure surfaces only when storage is actually touched.

Found by: a deploy-adjacent smoke test that exercised a real, unconfigured backend rather than the mocked test suite.

3. A stale-response race in file selection

Symptom: selecting file A, then selecting file B before A's request resolved, could let A's late-arriving response overwrite B's already-rendered content — the visibly selected file (B, highlighted) would silently show A's data.

Root cause: `App.tsx` held one shared async-loading instance for file content; nothing tracked which selection a given in-flight request belonged to, so whichever response arrived last won.

Fix: a generation token captured synchronously before the `await` on every `selectFile` call, checked against the current token after the response resolves — a stale result still populates the cache (so it's correct if reselected later) but never overwrites the currently-displayed selection. Verified with puppeteer by forcing A to resolve at 1500ms and B at 200ms and confirming B's data survived on screen.

4. Five separate local-vs-UTC date defects

The project's own contract treats "today" as a UTC calendar date (matching the backend's `datetime.now(timezone.utc).date()`), but at least five distinct places computed it from a local `Date` accessor or a raw instant comparison instead, across five separate fixes: the frontend's `validateInputs` originally compared an instant against local wall-clock time rather than UTC calendar dates; a related future-date check needed the same correction; `defaultRange()` built its end date from local calendar components; the *backend's own* `date.today()` read the server's local clock rather than UTC; and even after all of those fixes, `defaultRange()` and `validateInputs()` still disagreed for users east of UTC — so the app's own default date range failed its own validator on first render, with nothing typed, because `defaultRange` read the local calendar while the validator read UTC.

Fix: `defaultRange` now lives in `lib/transform.ts` next to `validateInputs`, takes the same injectable `now: Date = new Date()` clock, and reads only `getUTCFullYear / getUTCMonth / getUTCDate / setUTCDate`. The invariant `validateInputs(defaultRange(now), now) === null` is itself a test now, asserted at instants chosen because their local and UTC calendar dates differ, plus a month and a year boundary — so a regression that reintroduces a local-clock read fails a test rather than waiting for a user in the wrong timezone to hit it.

5. The chart palette was validated, not chosen by eye

The four line-series colors (max/min temperature, feels-like max/min) and the chart chrome, in both light and dark themes (`frontend/src/lib/theme.ts`), were run through a colorblind-safety check rather than picked visually. The worst adjacent pair under simulated color-vision deficiency measured a ΔE of 19.0 against a target minimum separation of 8 — comfortably clear — but the check also flagged that the light-mode "feels-like max" orange falls below a 3:1 contrast ratio against its background. That trade-off was accepted rather than hidden, which is exactly why `DataTable` sits beneath the chart as a mandatory element rather than an optional supplement: a reader who cannot reliably distinguish that series, or read it in bright light, can still read every value as a number.

6 Testing

116 backend tests across eight focused modules (`test_routes.py`, `test_schemas.py`, `test_storage.py`, `test_naming.py`, `test_open_meteo.py`, `test_geocoding.py`, `test_cors.py`, `test_errors.py`) and 53 frontend tests in a single pure-function suite (`lib/transform.test.ts`). Neither needs network access or credentials:

- Backend tests override `get_storage` with `InMemoryStorage` via FastAPI's dependency-override mechanism, and stub `httpx` at the transport layer with `respx` so no request reaches Open-Meteo's archive or geocoding APIs.
- Fake GCS/boto3 clients — injected through each storage class's optional `client` constructor parameter — exercise both `GCSStorage`'s and `S3Storage`'s real error-mapping paths (auth failures, missing objects, malformed JSON) without a cloud account.
- Frontend tests target the pure functions where off-by-one bugs actually live: pagination bounds (`clampPage`, and `paginate`'s guard against a negative page reinterpreting `Array.slice`'s semantics), the inclusive 31-day rule mirrored exactly from `schemas.py`, columnar-to-row transformation with nulls

preserved rather than dropped (`toDailyRows`), and the `defaultRange / validateInputs` UTC invariant from Finding 4.

There is no React component test harness. That is a deliberate scope decision, not an oversight: the brief's graded rules — validation, naming, the storage contract, the error envelope — all live in pure, directly-testable functions on both sides of the stack. Component-level rendering and interaction tests would add tooling and maintenance cost without covering additional graded behavior. The two defects that only manifest in an actual render — the stale-selection race and a handful of layout issues — were instead caught and verified with a puppeteer-driven manual pass, recorded in the ledger rather than encoded as an automated component suite.

7 Deployment

The backend runs as a non-root user inside a Docker container (Python 3.12 slim base); the frontend is a static Vite build. GCP/Cloud Run was the original target (see the design spec) but was abandoned before any deployment happened: activating Cloud Run and Cloud Build on a GCP project requires linking a billing account — a payment method — which was not available for this case study. The stack was pivoted to a no-card path: a Docker-hosted backend, Supabase Storage's S3-compatible API in place of GCS, and a static frontend host. The brief permits this — it names "GCS or S3" — and the pivot cost no route or schema changes, only the addition of `S3Storage` behind the existing Protocol.

Honest note: `DEPLOY.md`, as committed, documents the original Cloud Run + GCS runbook in full detail. It was written (commit `40f7ddc`) before the GCP-abandonment pivot and has not yet been rewritten for the Render/Supabase-Storage/Vercel path the project actually intends to use — that decision is recorded in the engineering ledger, not yet in the committed runbook. A reviewer following `DEPLOY.md` literally today would be deploying the abandoned path. This is flagged here rather than glossed over.

Backend and frontend URLs are inlined into the frontend bundle at build time (`VITE_API_BASE`), not read at runtime — changing the backend URL after a build requires a rebuild, not just an environment-variable edit on the host. If the chosen backend host's free tier sleeps the service on inactivity, the first request after an idle period pays a cold-start cost — commonly on the order of tens of seconds while the container boots — before the API responds normally again; subsequent requests are fast until the service idles out once more.

8 Known limitations

- **Second-granularity filenames.** The object-name timestamp is `%Y%m%dT%H%M%SZ` — seconds, not milliseconds. Two identical requests (same lat/lon/dates) completed within the same UTC second produce the same object name, and the second overwrites the first.
- **No authentication.** `POST /store-weather-data` is public; see the design-decisions section for what a production deployment would add first.
- **Recharts bundle size.** The production build's main chunk exceeds Vite's 500 KB warning threshold. It builds and runs fine; no code splitting has been added, since a single-page dashboard has nothing meaningful to split into a separate route.
- **Listing is unpaginated** at the API layer — see the corresponding design decision. Assumes bucket contents stay small.
- **The most recent day or two may return null readings** while Open-Meteo's ERA5 archive catches up to real time. The UI already renders a null as a gap in the chart and an em-dash in the table, so no special-

casing was added for this specific lag.

- `DEPLOY.md` is **stale** relative to the actual deployment plan — see the callout in the Deployment section above.

9 Running locally

Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate          # source .venv/bin/activate on macOS/Linux
pip install -r requirements-dev.txt
copy .env.example .env          # cp on macOS/Linux – edit as needed
uvicorn app.main:app --reload
pytest                          # 116 tests, no network, no credentials
```

Frontend

```
cd frontend
npm install
copy .env.example .env.local    # set VITE_API_BASE=http://localhost:8000
npm run dev
npm test                        # 53 tests
```

Running against a storage emulator (no cloud account needed)

Set `STORAGE_BACKEND=gcs` (or `s3`) and point the matching emulator variable at a local emulator instead of a real bucket, e.g. `STORAGE_EMULATOR_HOST=http://127.0.0.1:9023` in `backend/.env`.

Use **127.0.0.1**, not `localhost`. The emulator binds IPv4 only, and on Windows, `localhost` resolves to `:::1` (IPv6) first — every request then pays a failed IPv6 connection attempt before falling back to IPv4. Measured directly: 2.06s per call via `localhost` versus 0.009s via `127.0.0.1`, roughly 230× — which presents as the UI hanging on "Loading file..." when the actual cause is DNS/name-resolution ordering, not the emulator itself.